

Chapter 9 Weighted and Combined Models

The thought process around weighted and combined forecasting (also called **composite forecasting**) is not new. The topic has been studied for years, and empirical evidence indicates that the combination of forecast methods tends to outperform most single forecast methods.

It is better to average forecasts in hope that by so doing, the biases among the methods and/or forecasters will compensate for one another. As a result, forecasts developed using different methods are expected to be more useful in cases of high uncertainty. The method is especially relevant for long-range forecasting, where uncertainty is extremely high. Many different studies have been done to show the value of combining forecasts.

There are two basic weighting methods:

1. Simple Averaging of Forecasts
2. Weighted Methods Based on Error

The simple averaging approach is just as easy as it sounds. It takes the average of the forecasted values from each of the models being combined.

The code below just takes the average of the four models we have been working with for this dataset - the Holt-Winters exponential smoothing model, the dynamic regression ARIMA model, the neural network model, and Prophet algorithm model. We then calculate the MAE and MAPE from this model.

```
For.Avg <- (HWES.USAir.train$mean +
  forecast::forecast(Full.ARIMA, xreg = cbind(Sep11, Sep11.L1, Sep11.L2,
  Pass.Forecast +
  tail(predict(Prof, forecast.data)$yhat, 12))/4
```

```
Avg.error <- test - For.Avg
```

```
Avg.MAE <- mean(abs(Avg.error))
```

```
Avg.MAPE <- mean(abs(Avg.error)/abs(test))*100
```

```
Avg.MAE
```

```
## [1] 1039.604
```

```
Avg.MAPE
```

```
## [1] 1.576574
```

This average of the four models performs better than any of the individual models. However, maybe not all of the models are needed in this average. In fact, trying out many different combinations of models shows that only the exponential smoothing, neural network, and Prophet model are needed.

```
For.Avg <- (HWES.USAir.train$mean +
  Pass.Forecast +
  tail(predict(Prof, forecast.data)$yhat, 12)
)/3
```

```
Avg.error <- test - For.Avg
```

```
Avg.MAE <- mean(abs(Avg.error))
```

```
Avg.MAPE <- mean(abs(Avg.error)/abs(test))*100
```

```
Avg.MAE
```

```
## [1] 992.4662
```

```
Avg.MAPE
```

```
## [1] 1.502
```

The MAE and MAPE are lower than any of the individual models as well as the previous average across all models.

The second method uses weighted averaging instead of simple averaging. In weighted averaging, the analysts select the weights to assign to the individual forecasts. Typically, we will assign weights that minimize the variance of the forecast errors over the forecasted period. With only two forecasts, there is an easily derived mathematical solution to what these weights should be to minimize variance. However, three or more forecasts being combined makes the mathematical equations much harder. In these scenarios we can use linear regression to help. We run a regression of the actual values of Y against the forecasted values of Y to let the regression choose the optimal weight for each. To make sure that the weights sum to one, we can use some algebra. By setting the weight (coefficients) of one of the models to one using the `offset` function, we can then use the `I` function to estimate the weights (coefficients) on the differences between the remaining models and the offset

model. By working out the math algebraically, we can see that each model in an `I` function gets that weight, but the original model in the `offset` function gets the weight of 1 minus the remaining weights.

```
Pass.Fit.NN <- rep(NA, 207)
```

```
for(i in 25:207){
  Pass.Fit.NN[i] <- training[i - 24] + NN.Model$fitted[i - 12]
}
```

```
Pass.Fit.ARIMA <- Full.ARIMA$fitted
```

```
Pass.Fit.HWES <- HWES.USAir.train$fitted
```

```
Pass.Fit.Prophet <- head(predict(Prof, forecast.data)$yhat, 207)
```

```
## Warning in make_holidays_df(year.list, m$country_holidays):
```

```
## Holidays for US are only supported from 1995 to 2044
```

```
## Warning in make_holidays_df(year.list, m$country_holidays):
```

```
## Holidays for US are only supported from 1995 to 2044
```

```
WC.Model <- lm(training ~ offset(Pass.Fit.HWES) + I(Pass.Fit.ARIMA - Pass.Fit.HWES))
summary(WC.Model)
```

```
##
## Call:
## lm(formula = training ~ offset(Pass.Fit.HWES) + I(Pass.Fit.ARIMA -
##      Pass.Fit.HWES) + I(Pass.Fit.NN - Pass.Fit.HWES) + I(Pass.Fit.Prophet -
##      Pass.Fit.HWES) - 1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3157.8  -739.8   -43.1    729.2   3828.5
##
## Coefficients:
##                                     Estimate Std. Error
## I(Pass.Fit.ARIMA - Pass.Fit.HWES)    0.75022    0.06980
## I(Pass.Fit.NN - Pass.Fit.HWES)      -0.02466    0.02487
## I(Pass.Fit.Prophet - Pass.Fit.HWES)  0.28856    0.07249
##                                     t value Pr(>|t|)
## I(Pass.Fit.ARIMA - Pass.Fit.HWES)   10.747 < 2e-16 ***
## I(Pass.Fit.NN - Pass.Fit.HWES)      -0.992 0.322928
## I(Pass.Fit.Prophet - Pass.Fit.HWES)  3.981 0.000105 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1179 on 156 degrees of freedom
## (48 observations deleted due to missingness)
## Multiple R-squared:  0.6843, Adjusted R-squared:  0.6783
## F-statistic: 112.7 on 3 and 156 DF, p-value: < 2.2e-16
```

```
ARIMA.coef <- coef(WC.Model)[1]
NN.coef <- coef(WC.Model)[2]
Prophet.coef <- coef(WC.Model)[3]
HW.coef <- 1 - ARIMA.coef - NN.coef - Prophet.coef
```

Now we can just combine the forecasts with their respective weights to get our final forecast. From there we can evaluate the MAE and MAPE for this minimum variance weighted average forecast.

```
For.W.Avg <- HW.coef*HWES.USAir.train$mean +
  ARIMA.coef*forecast::forecast(Full.ARIMA, xreg = cbind(Sep11, Sep11.
  NN.coef*Pass.Forecast +
  Prophet.coef*tail(predict(Prof, forecast.data)$yhat, 12)
```

```
## Warning in make_holidays_df(year.list, m$country_holidays):
## Holidays for US are only supported from 1995 to 2044
```

```
## Warning in make_holidays_df(year.list, m$country_holidays):
## Holidays for US are only supported from 1995 to 2044
```

```
W.Avg.error <- test - For.W.Avg
```

```
W.Avg.MAE <- mean(abs(W.Avg.error))
```

```
W.Avg.MAPE <- mean(abs(W.Avg.error)/abs(test))*100
```

```
W.Avg.MAE
```

```
## [1] 1060.991
```

```
W.Avg.MAPE
```

```
## [1] 1.594627
```

Unfortunately, just because we have the minimum variance on the training data, doesn't mean that we are good on the test dataset. In fact, in practice, the regular average of forecasts is very hard to beat.

9.1 Python Code for Weighted and Combined Models

9.1.1 Simple Average

```
fcast_avg = (fcast + fcast2 + fcst['yhat'].tail(12) + fcast5)/4
```

```
error_avg = test['Passengers'] - fcast_avg
```

```
MAPE_avg = np.mean(abs(error_avg)/test['Passengers'])*100
```

```
MAPE_avg
```

```
## 1.148253225520028
```

```
fcast_avg2 = (fcast + fcst['yhat'].tail(12) + fcast5)/3

error_avg = test['Passengers'] - fcast_avg2
MAPE_avg = np.mean(abs(error_avg)/test['Passengers'])*100
MAPE_avg

## 1.1117330494732722
```